

Virtualização da CPU em Sistemas Operacionais

Esta seção aborda como o sistema operacional (SO) permite que múltiplos programas pareçam executar simultaneamente, mesmo com um número limitado de CPUs. Vamos explorar os mecanismos, desafios e técnicas envolvidas na virtualização da CPU.

Desafios da Virtualização da CPU

O SO deve compartilhar a CPU entre vários programas, criando a ilusão de execução simultânea. A ideia básica é o **time-sharing**, onde cada processo é executado por um curto período.

- **Performance:** Os mecanismos do SO não podem adicionar sobrecarga excessiva ao sistema.
- **Controle:** O SO deve manter o controle sobre a CPU, evitando que processos:
 - Tomem o controle da CPU (e.g., loops infinitos).
 - Acessem ou corrompam informações de outros programas na memória ou no disco.

Técnica Básica: Execução Direta Limitada

- **Objetivo:** Executar um programa na CPU o mais rápido possível (performance).
- **Solução:** O programa é executado diretamente no hardware da CPU (**execução direta**).
- **Problemas:** Como o SO garante que o processo não realize operações indesejadas e como interrompe o processo para alternar com outro?
 - É necessário limitar a execução dos programas, restringindo o que eles podem fazer.

Modos de Execução Restritos

Os processos de usuário não podem realizar operações restritas livremente (e.g., solicitações de E/S para o disco, acesso a recursos como CPU ou memória). O SO deve mediar essa interação com a ajuda do hardware. A CPU fornece dois modos de execução:

- **Modo Usuário:** O código em execução não pode realizar operações privilegiadas (e.g., um processo não pode fazer operações de E/S diretamente).
- **Modo Kernel:** O código em execução tem acesso total ao hardware e pode realizar qualquer operação restrita. O SO é executado neste modo.

O que acontece se um processo de usuário precisa realizar uma operação privilegiada? Aqui entram as **system calls**.

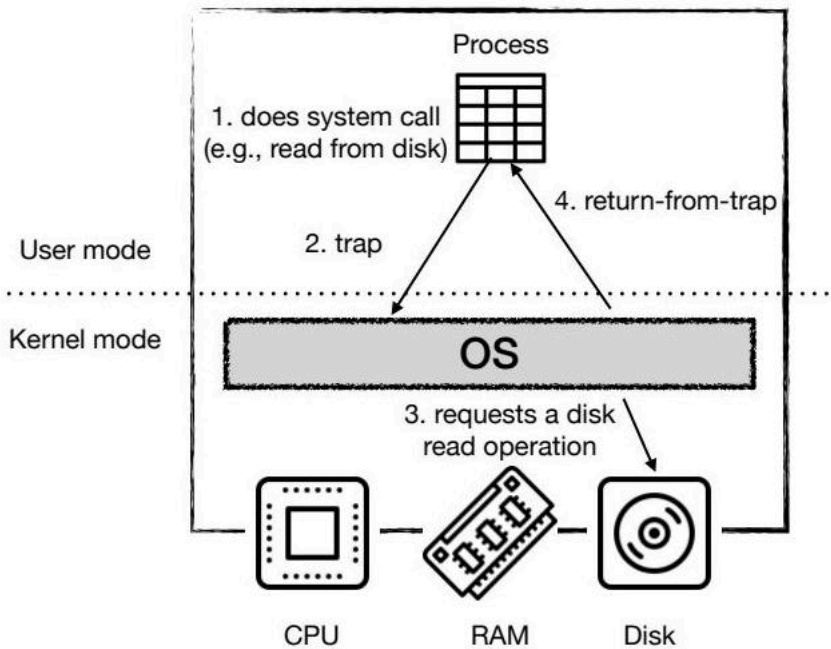
System Calls

System calls permitem que o kernel (SO) exponha operações privilegiadas aos processos de usuário (e.g., criação de processos, comunicação, alocação de memória, E/S).

System calls são solicitações de um programa de computador ao sistema operacional para executar uma tarefa que o programa não tem permissão para executar diretamente.

Para executar uma system call, o processo chama uma instrução especial (**trap**), elevando o nível de privilégio para o modo kernel e saltando para o código do SO (tratador de trap). O SO pode então realizar operações privilegiadas em nome do processo de usuário. Quando termina, o SO chama uma instrução de retorno de trap, que reduz o nível de privilégio para o modo usuário e retoma a execução do processo.

Aqui está um resumo visualizado no diagrama a seguir:



O diagrama ilustra o processo de uma chamada de sistema, mostrando a transição entre o modo usuário e o modo kernel. Ele detalha como um processo no modo usuário faz uma chamada de sistema, que aciona uma interrupção (trap) para mudar para o modo kernel. No modo kernel, o sistema operacional manipula a chamada de sistema e, em seguida, retorna para o modo usuário, permitindo que o processo continue sua execução.

Execução Direta Limitada (Atualizada)

O processo de execução direta limitada envolve a criação de uma entrada PCB, alocação de memória, carregamento do programa na memória e, em seguida, a transição para o modo de usuário para executar o programa. Se o programa precisar executar uma chamada de sistema, ele faz uma "trap" para o SO (modo kernel), que lida com a solicitação e retorna para o modo de usuário.

Troca de Modos: Traps e Interrupções

Duas formas de instruções levam à transferência do controle do usuário para o modo kernel:

- **Trap:** Gerado pelo processo atual em execução.
 - O programa de usuário solicita um serviço do SO (como vimos antes, e.g., system call).
 - O programa de usuário realiza uma ação ilegal (exceção, e.g., divisão por zero, acesso ilegal à memória).
- **Interrupção:** Causada por dispositivos e pode não estar relacionada com o processo em execução.
 - Quando um dispositivo (disco, teclado, placa de rede) requer atenção do SO (e.g., leitura do disco está pronta, há entrada do teclado para ser lida).

Interrupções: Por Que São Úteis?

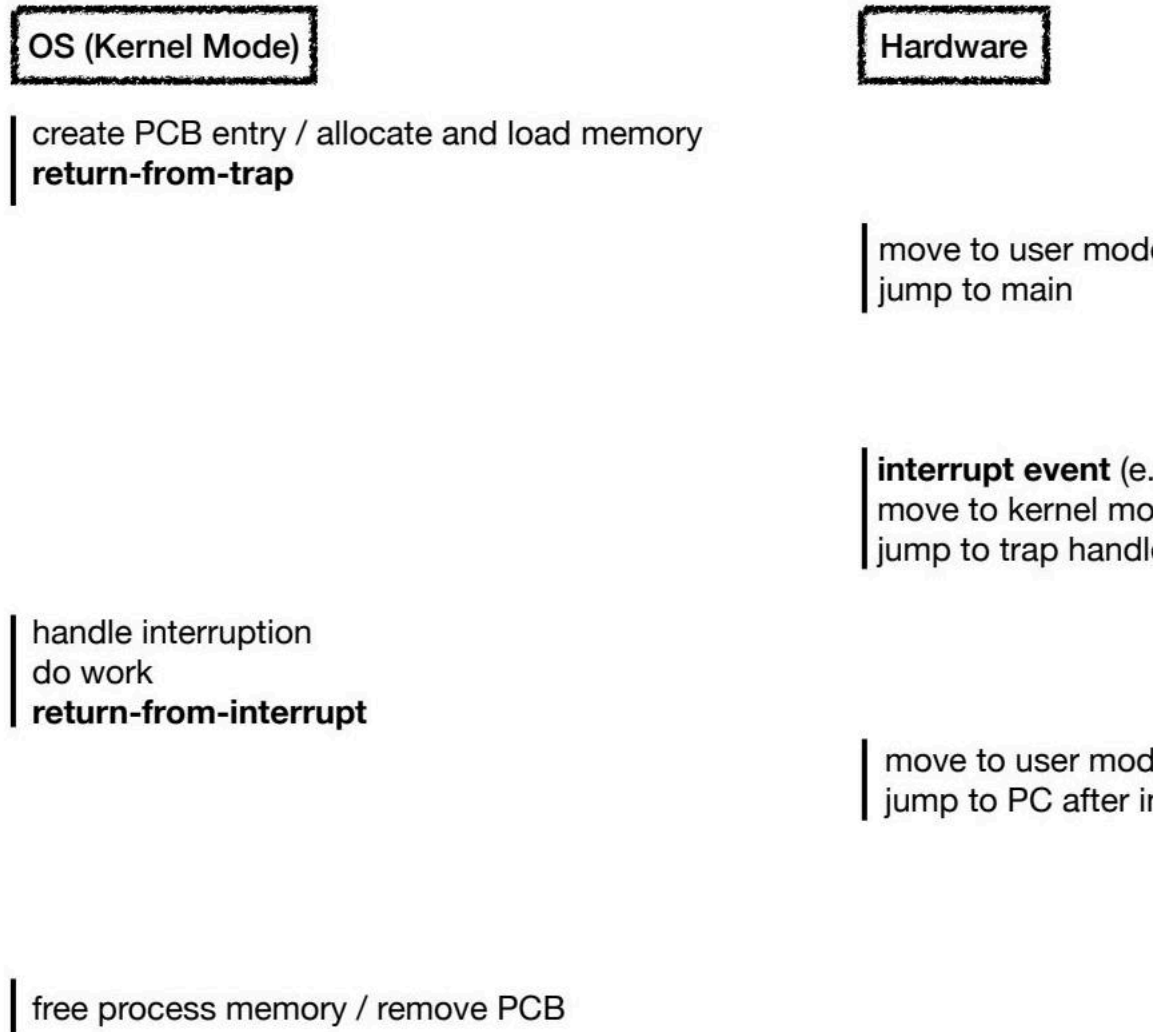
Imagine que um processo faz uma solicitação de leitura de um arquivo. O SO pede ao dispositivo de disco para fornecer o conteúdo. O disco leva um tempo para trazer a informação para a memória.

- O SO poderia verificar periodicamente (polling) o dispositivo de disco para saber se a solicitação foi concluída.
- No entanto, se os eventos demorarem muito para completar, o SO estaria desperdiçando ciclos de CPU preciosos que poderiam ser usados para executar outros processos!

As interrupções são geradas pelos dispositivos de forma assíncrona.

- **Benefício:** O SO é notificado quando os eventos ocorrem, liberando a CPU para outros processos.
- **Cuidado:** As interrupções são emitidas concorrentemente, enquanto outros processos ou o SO estão em execução na CPU.

O diagrama abaixo representa o tratamento de interrupções:



Este diagrama ilustra o processo de como as interrupções são tratadas em um sistema operacional, mostrando a interação entre o sistema operacional (executado no modo kernel), o hardware e os processos do usuário (executados no modo usuário). O diagrama destaca como um evento de interrupção de hardware força uma transição do modo de usuário para o modo kernel, onde o sistema operacional lida com a interrupção antes de retornar ao processo do usuário.

Estado do Processo: Traps e Interrupções

Tanto as instruções de trap quanto as de interrupção são assistidas por hardware. O SO não pode tomar a CPU de um processo sem ajuda. A CPU deve interromper o processo, mudar para o modo kernel, salvar algumas informações e saltar para o código do SO.

Quando um processo é interrompido, a CPU deve salvar um conjunto de **registradores**.

Registradores contêm informações (e.g., program counter) requeridas pela CPU para retomar a execução do processo quando o SO chama uma instrução de retorno de trap/interrupção.

Os registradores são armazenados, por exemplo, em uma **pilha kernel por processo**. No x86, o processador empurra o program counter, flags e outros registradores para a pilha kernel. A instrução de retorno de trap retira esses valores da pilha e retoma a execução do programa em modo usuário.

Tabela de Interrupção/Trap

No momento da inicialização, o kernel (SO) configura uma tabela de interrupção/trap. A tabela informa ao hardware qual código executar (tratador de interrupção/trap) sob eventos excepcionais.

Manipulador do SO	System Call 1	Manipulador do Teclado	Manipulador do Disco	...
-------------------	---------------	------------------------	----------------------	-----

A tabela mapeia interrupções e traps (e.g., identificados por um número) aos seus respectivos tratadores. Ela distingue interrupções de diferentes dispositivos (e.g., disco, teclado) e solicitações (e.g., leitura, escrita do disco). Ela também distingue entre diferentes tipos de trap (e.g., exceção, system call).

Integração com a Biblioteca C

Quando um programa chama uma system call (e.g., `fork()`, `open()`), uma chamada de procedimento para a biblioteca C é feita. A biblioteca usa uma convenção de chamada acordada com o kernel do SO. Os argumentos da system call e o número (identificador) são colocados em locais conhecidos (e.g., pilha, registradores especiais). A biblioteca executa a instrução de trap correspondente (system call). A tabela de interrupção/trap é usada para verificar o tratador de trap da system call. Ao executar o código do tratador de trap, o SO pode validar o número da system call e executar o código correspondente. Essa indireção proporciona segurança (mais seguro do que saltar diretamente para um endereço de memória para executar o código).

O Dilema: Troca de Processos

Como o SO pode compartilhar a CPU entre vários processos? Isso requer pausar um processo e executar outro. Mas vimos que, se um processo está em execução na CPU, o SO não está em execução. Como o SO pode retomar o controle para agendar outro processo?

Abordagem Cooperativa: Esperar por System Calls

Quando um processo faz uma system call, o kernel do SO retoma o controle para:

- Lidar com a system call (e.g., pedir ao disco para ler algum conteúdo).
- Fazer outras tarefas, como agendar outro processo!

O SO também pode retomar o controle sob interrupções de dispositivos e outras traps (exceções). Em uma abordagem cooperativa, o SO assume que os processos irão:

- Emitir system calls (e.g., bloquear para fazer E/S no disco).
- Ou voluntariamente ceder a CPU ao SO (a system call `yield` serve exatamente para isso).

E se tivermos programas maliciosos ou com bugs (e.g., loops infinitos)?

Interrupção do Timer: Abordagem Não Cooperativa

Precisamos de ajuda do hardware novamente, especificamente de uma interrupção de timer.

- E.g., a cada X milissegundos, uma interrupção é gerada pelo hardware.
- O processo é interrompido, um tratador de interrupção específico é executado e o SO ganha o controle!

O SO é responsável por:

- Definir o código para o tratador de interrupção do timer.
- Configurar o intervalo do timer (no momento da inicialização).

Lembre-se: quando a interrupção ocorre, o estado do processo deve ser salvo para que o processo possa ser retomado mais tarde.

Troca de Contexto: Salvando e Restaurando o Contexto

Quando o SO retoma o controle (cooperativamente ou não cooperativamente):

- Ele deve decidir se continua executando o mesmo processo ou executa outro.
- Essa decisão é tomada pelo **scheduler** do SO.

Quando o scheduler decide trocar o processo A pelo processo B, uma **troca de contexto** (context switch) é feita.

- O SO deve salvar informações do processo A (e.g., program counter, limites de memória, ponteiro da pilha kernel) em sua estrutura PCB (Process Control Block).
- Troca de contextos, mudando o ponteiro da pilha da CPU (um registrador) para a pilha kernel do processo B.

Execução Direta Limitada (Troca de Contexto)

O processo de troca de contexto envolve salvar o estado do processo atual (A) e restaurar o estado do próximo processo (B) a ser executado. Isso inclui salvar os registradores, o program counter e outras informações de contexto na PCB do processo A, e restaurar o estado do processo B a partir de sua PCB.

Performance da Troca de Contexto

Em 1996, ao executar o Linux 1.3.37 em uma CPU P6 de 200 MHz, as system calls levavam aproximadamente 4 microssegundos e uma troca de contexto aproximadamente 6 microssegundos. Os sistemas modernos têm um desempenho quase uma ordem de magnitude melhor, com resultados abaixo de microssegundos em sistemas com processadores de 2 ou 3 GHz.

Sempre que uma system call para o disco é feita, por exemplo, o SO provavelmente agendará outro processo para ser executado. Por quê? Qual o impacto de implementar um programa que faz system calls repetitivas que levam poucos microssegundos?